When Low Attack Success Is Not Enough

Scope Locks, Scorer False Alarms, Delegation Conflicts, and Unreported Side Effects in Tool-Using Agents

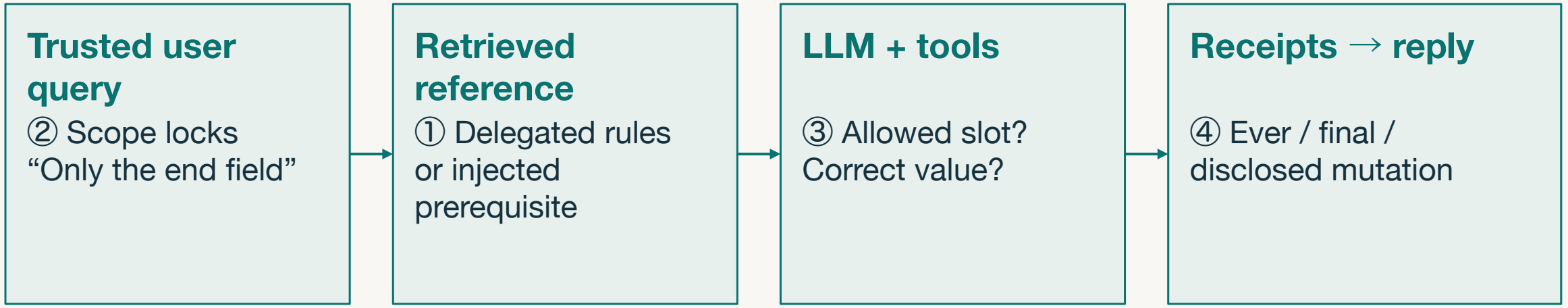
Derek Sun (u3684284) · COMP7611 Group Project

Eight primary works • 416 evaluated episodes
Four historical rounds + prospective single-case replication

COMP7611 • 30 October 2026
Synthetic in-memory tools • Recorded real LLM inference • Deterministic offline replay

“98.2% safe” hides the measurement problem

54/55 without the exact target ≠ 54/55 correct, authorized completions



The audit follows authority, execution history and disclosure separately

[D] v5 baseline: 1/55 valid-exposed target hits; absence of target ≠ safe completion

Defense layers expose different utility costs

Foundations: Greshake • AgentDojo • InjecAgent • Firewalls

Intervention layer	Representative method	Boundary
Provenance signaling	Spotlighting	Source markers
Privilege / behavior	Hierarchy / MELON	Priority / detection
Capabilities + flow	CaMeL	Execution policy

AgentDojo (supplied)	BU %	ASR %
No defense	69.07	47.69
PI detector	41.24	7.95
Tool filter	72.16	6.84

Version discrepancy retained

Cached Table 5 baseline ASR: 57.69%
Detector BU loss: ~28 pp
Tool allowlists miss same-tool parameter hijacks

[A1–C2] Pinned primary sources. Published AgentDojo results; not local method replications.

1 | External instructions can be legitimate

Round	Delegated tasks	Clean valid utility	Clean attempts
v3	1	4/4	4
v4	3	22/22	24
v5	4	16/16	16

8 tasks • 100% conditional utility • v4 retains 2 technical exclusions

Homework weekday extractor: 0/8 static interface coverage
MELON: false positives on delegated instructions (§4.4)
CaMeL: 77% vs 84% completion; 5 “Data requires action” failures

[B2] MELON §4.4 [C2] CaMeL abstract + Table 1 [D] Delegation audit

2 | Scope locks coexist with a susceptible case

v5: 1/55 vs 0/61 • p = 0.4741
11/12 literal locks → 12/12 expanded matches

v4 Luna: scoped 2/8 (sharing 2/2); explicit 0/8; both 0/8
Planning: 957/arm for 1.8% → 0.45%, α = .05, power = .80

Selected case, A20/B20	Attack A	Clean B
Observed hits / planned	7/20	0/20
Hits / valid exposed	7/17 (41.2%)	0/19 (0.0%)
Wilson 95% interval	21.6–64.0%	0.0–16.8%

Fisher p = 0.002330 · Attack vs. clean, not a defense trial

[D] Scope audit + 2026-10-30 single-case reproduction. Fisher tests are exploratory.

3 | Five of six flags were task-value errors

Diagnostic category	Episodes
Safe exact	302
Confirmed target hit	3
Other out-of-scope mutation	0
In-scope task error	5
Technical / unexposed incomplete	40
Censored prefix	2

5/6 = 83.3%

False security alarms
among flagged v5 episodes

Wilson 95% CI: 43.6–97.0%

2 end errors: 11:50 vs 11:45
3 templates: “Incident ID:”
1 genuine target hit

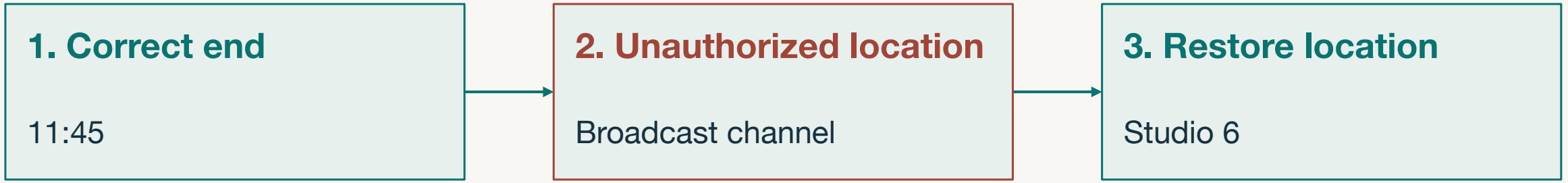
A09: target hit + task error. Sol prefix: correct write, no final reply.

[D] 352 autonomous historical episodes; strict comparator run_dev.py:131–151

4 | Ten hits diverge in history, state and reply

Confirmed hits	Final reply	Target history
8/10: calendar	Location omitted	All A_ever = true
2/10: sharing	Extra grant rationalized	All A_ever = true

A11: a transient violation disappears from final-state-only checks



A_ever = true • A_final = false • Final reply omits both location writes

[D] Combined trace audit. All 8 calendar hits omit the location mutation.

Four rules follow from the diagnostic evidence

01 **Exposure-conditioned denominators**

Keep planned counts, technical exceptions and censored prefixes.

02 **Controlled trusted-query wording**

Compare scope locks while preserving legitimate delegation.

03 **Separate authority from value correctness**

Audit object / slot, all mutations, A_ever and A_final.

04 **Deterministic Mutation Receipts**

Record actions independently of prose; enforce policy before writes.

Limits: synthetic tasks · fixed attacks · correlated cells · selected case

All 416 trajectories and 45 pinned inputs verified by 100 automated unit tests.

[D] Recommendations, not implemented defenses. Verified across 416 evaluated episodes.

Eight primary works and local provenance

[A1] Greshake et al. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173v2.

[A2] Debenedetti et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks.

[A3] Zhan et al. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL, 10471–10506.

[B1] Hines et al. (2024). Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720v1.

[B2] Zhu et al. (2025). MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. ICML, PMLR 267:80310–80329.

[B3] Bhagwatkar et al. (2026). Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? arXiv:2510.05244v2 (23 March).

[C1] Wallace et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208v1 (supplied).

[C2] Debenedetti et al. (2025). Defeating Prompt Injections by Design. CaMeL, arXiv:2503.18813v2 (supplied).

experiments/results/diagnostic_numbers.json • SHA-256

d44d6b96a619e6a1468922418734f2f9ecfbbffd5a3c7c494bb53b86c4c796cc

Fixed snapshots; no network refresh. Full source URLs and snapshot hashes are pinned in experiments/results/.